



Jurusan Informatika

Scope in Python



Algoritma dan Pemrograman II

Team Teaching:
Lia Farhatuaini, M.Kom
Sokid, M.Kom

SCOPE-1

- Scope (indo: cakupan) dari sebuah nama (misalkan nama variabel) merupakan bagian dari kode dimana nama tersebut dikenali.
- Contoh: sebuah variabel yang berada di dalam fungsi memiliki scope hanya di dalam fungsi itu sendiri, karena parameter fungsi tidak dapat diakses dari luar fungsi

```
1  def penjumlahan(x):  
2      bilangan = 7  
3      return x + 7  
4  
5  print(penjumlahan(4))  
6  print(bilangan)
```

SCOPE-2

- Contoh: sebuah variabel yang berada di luar fungsi memiliki scope ke dalam fungsi (contoh 1) kecuali fungsi tersebut mendefinisikan sebuah variabel dengan nama yang sama (contoh 2 & 3)

```
1  # Contoh 1
2  bilangan = 2
3  def perkalian_bilangan(x):
4      return x*bilangan
5
6  print(perkalian_bilangan(7))
```

```
1  # Contoh 2
2  def perkalian_bilangan(x):
3      bilangan = 5
4      return x*bilangan
5
6  print(perkalian_bilangan(7))
```

```
1  # Contoh 3
2  def perkalian_bilangan(x):
3      bilangan = 7
4      return x*bilangan
5
6  bilangan = 3
7  print(perkalian_bilangan(7))
```

SCOPE-3

- kita bisa memberikan keyword '**global**' kepada variabel supaya variabel tersebut memiliki scope global (bisa diakses dari mana saja)

```
1  bilangan = 2
2  print(bilangan)
3
4  def return_bilangan():
5      global bilangan
6      bilangan = 5
7      return bilangan
8
9  print(return_bilangan())
10 print(bilangan)
```

Multi-Parameter Function in Python



Algoritma dan Pemrograman II

Team Teaching:
Lia Farhatuaini, M.Kom
Sokid, M.Kom

KUIS IMT



IAIN SYEKH NURJATI CIREBON
PROGRAM STUDI INFORMATIKA

- buatlah sebuah program python yang memiliki fungsi untuk menghitung massa index tubuh (IMT) dengan ketentuan:
 - user memasukkan berat dalam kilogram dan tinggi badan dalam meter
 - lebih lanjut lengkapi salinan kode pada slide selanjutnya

IMT = $\frac{\text{Berat Badan (Kg)}}{(\text{Tinggi Badan (m)} \times \text{Tinggi Badan (m)})}$

Ukuran Lingkar Perut
Laki - Laki : ≥ 90 cm
Perempuan : ≥ 80 cm

Hasil Perhitungan IMT
Normal : 18,5 - 25,0
Gemuk : $> 25,0 - 27,0$
Obesitas : $> 27,0$

The infographic includes an illustration of a woman standing on a scale and a close-up of a waist measurement tape.

KUIS IMT

```
1 def hitung_imt(berat, tinggi):
2     ... #hitung nilai imt
3     return imt
4
5 berat = ...
6 tinggi = ...
7
8 index_massa_tubuh = hitung_imt(berat, tinggi)
9 kategori = ["Normal", "Gemuk", "Obesitas"]
10 #ketegorikan nilai imt yg sudah di dapat
11 if ....:
12     print("Index massa tubuh anda adalah ", index_massa_tubuh, "termasuk kategori ", kategori[0])
13 elif ....:
14     print("Index massa tubuh anda adalah ", index_massa_tubuh, "termasuk kategori ", kategori[1])
15 else: print("Index massa tubuh anda adalah ", index_massa_tubuh, "termasuk kategori ", kategori[2], ". Anda harus diet!")
```

FUNGSI SEGI TIGA-1

- misalkan kita ingin membuat fungsi untuk mengecek apakah 3 nilai panjang dari suatu sisi dapat membentuk segi tiga atau tidak.
- kita tahu bahwa untuk membentuk suatu segi tiga yaitu jika kedua sisi yang dijumlahkan harus lebih besar dengan sisi yang ke-3.

```
1  def cek_segitiga(a,b,c):
2      if a + b <= c:
3          return False
4      if b + c <= a:
5          return False
6      if c + a <= b:
7          return False
8      return True
9
10 print(cek_segitiga(1,1,1))
11 print(cek_segitiga(1,1,3))
```


FUNGSI SEGI TIGA-2

- fungsi segi tiga tadi bisa kita persingkat menjadi:

```
1 def cek_segitiga(a,b,c):
2     if a + b <= c or b + c <= a or c + a <= b:
3         return False
4     return True
5
6 print(cek_segitiga(1,1,1))
7 print(cek_segitiga(1,1,3))
```

- bisa disingkat lagi menjadi:

```
1 def cek_segitiga(a,b,c):
2     return a + b > c and b + c > a and c + a > b
3
4 print(cek_segitiga(1,1,1))
5 print(cek_segitiga(1,1,3))
```

KUIS FAKTORIAL

- faktorial adalah sebagai berikut: lanjutkan ... dalam kode

```
1  def faktorial(n):
2      #bilangan yang akan difaktorial harus lebih besar dari 0
3      if n < 0:
4          return None
5      # 0! dan 1! nilainya sama (1)
6      if n < 2:
7          return 1
8
9      hasil = 1
10     for i in range (...):
11         ...
12     return ...
13
14 n = int(input("Masukkan nilai yang ingin di faktorial: "))
15 print(n, "! = ", faktorial(n))
```

0! = 1 (yes! it's true)

1! = 1

2! = 1 * 2

3! = 1 * 2 * 3

4! = 1 * 2 * 3 * 4

:

:

n! = 1 * 2 * 3 * 4 * ... * n-1 * n

KUIS FIBONACCI

- bilangan fibonacci merupakan sebuah deret dari bilangan integer dengan ketentuan:
 - elemen pertama dan kedua dari deret = 1 (Fib1=1, Fib2=1)
 - bilangan ke-3 dan seterusnya merupakan hasil penjumlahan dari 2 bilangan sebelumnya (Fib3 = Fib2 + Fib 1, Fib4= Fib3 + Fib 2, dst..)

```
fib_1 = 1 fib_2 = 1 fib_3 = 1 + 1 = 2 fib_4 = 1 + 2 = 3 fib_5 = 2 + 3 = 5 fib_6 = 3 + 5 = 8 fib_7 = 5 + 8  
= 13
```

KUIS FIBONACCI

- lengkapi kode pada bagian ...

```
1  def fibonacci(n):
2      if n < 1:
3          return None
4      if n < 3:
5          return 1
6
7      elem_1 = elem_2 = 1
8      hasil_jumlah = 0 #untuk menampung hasil dari penjumlahan 2 elemen
9      for i in range(...):
10         ... #Proses jumlah
11         ... #Tukar elemen
12     return ...
13
14 #test
15 for i in range(1,10):
16     print(n, "->", fibonacci(n))
```


REKURSIF-1

- rekursif merupakan suatu teknik pemrograman dimana fungsi memanggil dirinya sendiri
- sebuah fungsi yang di dalamnya ada pemanggilan kepada dirinya sendiri merupakan fungsi rekursif.
- dengan menggunakan fungsi rekursif kode kita menjadi lebih ringkas dan jelas.
- namun, perlu berhati-hati dalam menggunakan fungsi rekursif sebab kita bisa saja membuat fungsi rekursif tersebut masuk ke dalam infinite looping atau tidak pernah berhenti/terminate.
- dengan menggunakan fungsi rekursif, coding kita lebih efisien (running timenya lebih cepat) tetapi pemanggilan fungsi rekursif juga membutuhkan banyak memory.

REKURSIF-2

- untuk lebih jelas dalam memahami fungsi rekursif kita gunakan 2 kasus sebelumnya yaitu faktorial dan fibonacci:

```
def fibonacci(n):  
    if n < 1:  
        return None  
    if n < 3:  
        return 1  
    return fibonacci(n-1) + fibonacci(n-2)  
  
def faktorial(n):  
    if n < 0:  
        return None  
    if n < 2:  
        return 1  
    return n * faktorial(n-1)
```